

8. Multimodal Table Scanning Voice Interaction

8. Multimodal Table Scanning Voice Interaction

1. Concept Introduction
 - 1.1 What is "Multimodal Table Scanning"?
 - 1.2 Brief Description of Implementation Principles
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largemodel/utis/tools_manager.py`)
 2. Model Interface Layer (`largemodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring Offline Large Model
 - 3.1.1 Configuring LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuring Model Interface (`large_model_interface.yaml`)
 - 3.2 Startup and Testing

1. Concept Introduction

1.1 What is "Multimodal Table Scanning"?

Multimodal Table Scanning is a technology that uses image processing and artificial intelligence techniques to identify and extract table information from images or PDF documents. It not only focuses on visual table structure recognition, but also integrates multimodal data such as text content and layout information to enhance understanding of tables. **Large Language Models (LLMs)** provide powerful semantic analysis capabilities for understanding the extracted information, complementing each other and jointly improving the intelligence level of document processing.

1.2 Brief Description of Implementation Principles

1. Table Detection and Content Recognition

- Use computer vision techniques to locate tables in documents, and convert the text within tables into editable format through OCR technology.
- Use deep learning methods to parse table structure (row/column division, merged cells, etc.) and generate structured data representation.

2. Multimodal Fusion

- Integrate visual information (such as table layout), text (OCR results), and any existing metadata (such as file type, source) to form a comprehensive data view.
- Use specially designed multimodal models (such as LayoutLM) to simultaneously process these different types of data for more accurate understanding of table content and contextual relationships.

2. Code Analysis

Key Code

1. Tool Layer Entry (largemodel/utils/tools_manager.py)

The `scan_table` function in this file defines the execution flow of the tool, particularly how it constructs a Prompt requesting Markdown format output.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def scan_table(self, args):
        """
        Scan a table from an image and save the content as a Markdown file.
        """
        self.node.get_logger().info(f"Executing scan_table() tool with args:
{args}")
        try:
            image_path = args.get("image_path")
            # ... (Path check and fallback)

            # Construct a prompt asking the large model to recognize the table
            and return it in Markdown format.
            if self.node.language == 'zh':
                prompt = "请仔细分析这张图片，识别其中的表格，并将其内容以Markdown格式返
回。"
            else:
                prompt = "Please carefully analyze this image, identify the
table within it, and return its content in Markdown format."

            result = self.node.model_client.infer_with_image(image_path, prompt)

            # ... (Extract Markdown text from result)

            # Save the recognized content to a Markdown file.
            md_file_path = os.path.join(self.node.pkg_path, "resources_file",
"scanned_tables", f"table_{timestamp}.md")
            with open(md_file_path, 'w', encoding='utf-8') as f:
                f.write(table_content)

            return {
                "file_path": md_file_path,
                "table_content": table_content
            }
        # ... (Error handling)
```

2. Model Interface Layer

(largemodel/utils/large_model_interface.py)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
```

```

        # ... (Prepare messages)
        try:
            # Based on the value of self.llm_platform, determine which specific
            implementation to call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Tongyi model logic
                pass
            # ... (Other platform logic)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

Table scanning is a typical application that converts unstructured image data into structured text data. Its core technology is still **guiding model behavior through Prompt Engineering**.

1. Tool Layer (`tools_manager.py`):

- The `scan_table` function is the business process controller for this feature. It receives an image containing a table as input.
- The most critical operation of this function is **constructing a goal-oriented Prompt**. This Prompt directly instructs the large model to perform two tasks: 1. Recognize the table in the image. 2. Return the recognized content in Markdown format. This mandatory requirement for output format is key to achieving unstructured-to-structured conversion.
- After constructing the Prompt, it calls the model interface layer's `infer_with_image` method, passing both the image and this formatted instruction.
- After receiving the returned Markdown text from the model interface layer, it performs a file operation: writing this text content to a new `.md` file.
- Finally, it returns structured data containing the new file path and table content.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function continues to serve as the unified "dispatch center". It receives the image and Prompt from `scan_table` and dispatches the task to the correct backend model implementation based on the current system configuration (`self.llm_platform`).
- Regardless of what the backend model is, this layer's task is to handle communication details specific to the platform, ensure that image and text data are sent correctly, and then return the plain text returned by the model (in this case, Markdown format text) to the tool layer.

In summary, the general flow of table scanning is: `ToolsManager` receives an image and constructs an instruction to "convert the table in this image to Markdown" -> `ToolsManager` calls the model interface -> `model_interface` packages the image and instruction and sends them to the corresponding model platform based on configuration -> The model returns Markdown format text -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` saves the text as a `.md` file and returns the result. This flow demonstrates how to use the large model's format-following capability as a powerful OCR (Optical Character Recognition) and data structuring tool.

3. Practical Operations

3.1 Configuring Offline Large Model

3.1.1 Configuring LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. **Open the file in terminal:**

```
vim ~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

2. **Modify/confirm `llm_platform`:**

```
model_service:                                #Model server node parameters
  ros__parameters:
    language: 'zh'                             #Large model interface language
    useolinetts: True                         #This setting is invalid in text
    mode, can be ignored

    # Large model configuration
    llm_platform: 'ollama'                    # Critical: Ensure this is set to
    'ollama'
```

3.1.2 Configuring Model Interface (`large_model_interface.yaml`)

This file defines which visual model is used when the platform is set to `ollama`.

1. Open the file in terminal

```
vim
~/yahboom_ws/install/largemodel/share/largemodel/config/large_model_interface.
yaml
```

2. Find the ollama-related configuration

```
#.....
## Offline Large Language Models
# Ollama configuration
ollama_host: "http://localhost:11434" # ollama server address
ollama_model: "llava"                # Critical: Change this to your downloaded
multimodal model, such as "llava"
#.....
```

Note: Please ensure the model specified in the configuration parameters (such as `llava`) can handle multimodal input.

3.2 Startup and Testing

1. **Prepare image file:**

Place a test image file at the following path:

```
/home/sunrise/yahboom_ws/src/largemodel/resources_file/scan_table
```

Then name the image `test_table.jpg`

2. **Start the `largemodel` main program:**

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake up:** Say to the microphone: "Hello, yahboom."
- **Conversation:** After the speaker responds, you can say: `analyze the table content`
- **Observe logs:** In the terminal running the `launch` file, you should see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, invokes the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker, and find an MD document recording the table information under the path `/home/sunrise/yahboom_ws/src/largemodel/resources_file/scan_table`.